

一.什么是树型动态规划

顾名思义，树型动态规划就是在“树”的数据结构上的动态规划，平时作的动态规划都是线性的或者是建立在图上的，线性的动态规划有二种方向既向前和向后，相应的线性的动态规划有二种方法既顺推与逆推，而树型动态规划是建立在树上的，所以也相应的有二个方向：

1 根—>叶：不过这种动态规划在实际的问题中运用的不多，也没有比较明显的例题，所以不在今天讨论的范围之内。

2 叶—>根：既根的子节点传递有用的信息给根，完后根得出最优解的过程。这类的习题比较多，下面就介绍一些这类题目和它们的一般解法。

二.例题与解析

加分二叉树

【问题描述】

设一个 n 个节点的二叉树 **tree** 的中序遍历为 $(1,2,3,\dots,n)$ ，其中数字 $1,2,3,\dots,n$ 为节点编号。每个节点都有一个分数（均为正整数），记第 i 个节点的分数为 di ，**tree** 及它的每个子树都有一个加分，任一棵子树 **subtree**（也包含 **tree** 本身）的加分计算方法如下：

subtree 的左子树的加分 $\times$ **subtree** 的右子树的加分 $+$ **subtree** 的根节点的分数

若某个子树为空，规定其加分为 1，叶子的加分就是叶节点本身的分数。不考虑它的空子树。

试求一棵符合中序遍历为 $(1,2,3,\dots,n)$ 且加分最高的二叉树 **tree**。要求输出：

- (1) **tree** 的最高加分
- (2) **tree** 的前序遍历

【输入格式】

第 1 行：一个整数 n ($n < 30$)，为节点个数。

第 2 行： n 个用空格隔开的整数，为每个节点的分数（分数 < 100 ）。

【输出格式】

第 1 行：一个整数，为最高加分（结果不会超过 4,000,000,000）。

第 2 行： n 个用空格隔开的整数，为该树的前序遍历。

【输入样例】

```
5
5 7 1 2 10
```

【输出样例】

```
145
3 1 2 4 5
```

算法分析：

法一：

因为二叉树的中序遍历已经给出，我们可以用动态规划来解决。本题主要考察二叉树的遍历和动态， $f[l,j]$ 表示包含结点 l 到 j 的二叉树的最大加分。

状态转移方程：

$f[l,j]=1(l>j)$

$f[l,j]=a[l](l=j)$

$f[l,j]=\max\{f[l,k-1]*f[k+1,j]+a[k]\}(l \leq k \leq j)$

此题在求最大加分的同时，还要求最大加分的二叉树的前序遍历。我们可以借助一个树组 G。

$g[l,j]$ 表示包含结点 l 到 j 的二叉树取得最大加分时的根结点的编号。

$G[l,j]=k$ （满足 $f[l,k-1]*f[k+1,j]+a[k]=f[l,j]$ ）。其实就是 $f[l,j]$ 取最大值时 k 的值。

构造最优解的形成过程（递归）：

get[l,j]用于输出包含结点 l 到 j 的二叉树取得最大加分时的前序遍历。

K:=g[l,j];

Write(k);{前序遍历先输出根结点}

If (k-1>=l) then begin write(' ');get(l,k-1);end;{输出左子树}

If (k+1<=j) then begin write(' ');get(k+1,j);end;{输出右子树}

参考程序：

{ \$n+ }

const maxn=30;

var n,l,j,k:integer;f:array[0..maxn,0..maxn] of double;

g:array[0..maxn,0..maxn] of integer;temp:double;

procedure out(l,j:integer);

var k:integer;

begin

k:=g[l,j];write(k);

if (k-1>=l) then begin write(' ');out(l,k-1);end;

if (k+1<=j) then begin write(' ');out(k+1,j);end;

end;

begin

assign(input,'tree.in');reset(input);

assign(output,'tree.out');rewrite(output);

read(n);

for l:=1 to n+1 do f[l,l-1]:=1;

for l:=1 to n do

begin read(f[l,i]);g[l,i]:=l; end;

for l:=n downto 1 do

for j:=l+1 to n do

begin

f[l,j]:=0;

for k:=l to j do

begin

temp:=f[k,k]+f[l,k-1]*f[k+1,j];

if (temp-1e-3>f[l,j]) then begin f[l,j]:=temp;g[l,j]:=k;end;

end;

end;

writeln(f[1,n]:0:0);out(1,n);writeln;close(input);close(output);

end.

法二：[分析]很显然，本题适合用动态规划来解。如果用数组 $value[i,j]$ 表示从节点 i 到节点 j 所组成的二叉树的最大加分，则动态方程可以表示如下：

$$value[i,j]=\max\{value[i,i]+value[i+1,j],value[i+1,i+1]+value[i,i]*value[i+2,j],$$
$$value[i+2,i+2]+value[i,i+1]*value[i+3,j],\dots,value[j-1,j-1]+value[i,j-2]*value[j,j],$$
$$value[j,j]+value[i,j-1]\}$$

题目还要求输出最大加分树的前序遍历序列，因此必须在计算过程中记下从节点 i 到节点 j 所组成的最大加分二叉树的根节点，用数组 $root[i,j]$ 表示

[PASCAL 源程序]

{ \$N+ }

program NOIP2003_3_Tree;

const

 maxn=30;

var

 i,j,n,d:byte;

 a:array[1..maxn]of byte;

 value:array[1..maxn,1..maxn]of comp;

 root:array[1..maxn,1..maxn]of byte;

 s,temp:comp;

 f1,f2:text;fn1,fn2,fileNo:string;

procedure preorder(p1,p2:byte);{按前序遍历输出最大加分二叉树}

begin

 if p2>=p1 then begin

 write(f2,root[p1,p2], ' ');

 preorder(p1,root[p1,p2]-1);

 preorder(root[p1,p2]+1,p2);

 end;

end;

begin

 write('Input fileNo:');readln(fileNo);

 fn1:='tree.in'+fileNo;fn2:='tree.ou'+fileNo;

 assign(f1,fn1);reset(f1);

 assign(f2,fn2);rewrite(f2);

 readln(f1,n);

 for i:=1 to n do read(f1,a[i]);

 close(f1);

 fillchar(value,sizeof(value),0);

 for i:=1 to n do begin

 value[i,i]:=a[i];{计算单个节点构成的二叉树的加分}

 root[i,i]:=i;{记录单个节点构成的二叉树的根节点}

 end;

 for i:=1 to n-1 do begin

 value[i,i+1]:=a[i]+a[i+1];{计算相邻两个节点构成的二叉树的最大加分}

 root[i,i+1]:=i;{记录相邻两个节点构成的二叉树的根节点；需要说明的是，两个节点构成的二叉树，其根节点可以是其中的任何一个；这里选编号小的为根节点，则编号大的为其

右子树；若选编号大的为根节点，则编号小的为其左子树；因此，最后输出的前序遍历结果会有部分不同，但同样是正确的。如果 最大加分二叉树的所有节点的度数都是 0 或 2，则最后输出的前序遍历结果是唯一的。}

```

end;
for d:=2 to n-1 do begin{依次计算间距为 d 的两个节点构成的二叉树的最大加分}
  for i:=1 to n-d do begin
    s:=value[i,i]+value[i+1,i+d];{计算以 i 为根节点，以 i+1 至 i+d 间所有节点为右子树
    的二叉树的最大加分}
    root[i,i+d]:=i; {记录根节点 i}
    for j:=1 to d do begin
      temp:=value[i+j,i+j]+value[i,i+j-1]*value[i+j+1,i+d];{计算以 i+j 为根节点，以 i 至
      i+j-1 间所有节点为左子树，以 i+j+1 至 i+d 间所有节点为右子树的二叉树的最大加分}
      if temp>s then begin{如果此值为最大}
        s:=temp;root[i,i+d]:=i+j;{记下新的最大值和新的根节点}
      end;
    end;
    temp:=value[i,i+d-1]+value[i+d,i+d];{计算以 i+d 为根节点，以 i 至 i+d-1 间所有节
    点为左子树的二叉树的最大加分}
    if temp>s then begin
      s:=temp;root[i,i+d]:=i+d+1;
    end;
    value[i,i+d]:=s;
  end;
end;
writeln(f2,value[1,n]:0:0);{输出最大加分}
preorder(1,n);{输出最大加分二叉树的前序遍历序列}
close(f2);
end.

```

[点评]基本题。考查了二叉树的遍历和动态规划算法。难点在于要记录当前最大加分二叉树的根节点。疑点是最大加分二叉树的前序遍历序列可能不唯一。

Ural 1018 二*苹果树

题目

有一棵苹果树，如果树枝有分叉，一定是分 2 叉（就是说没有只有 1 个儿子的结点）这棵树共有 N 个结点（叶子点或者树枝分叉点），编号为 $1-N$ ，树根编号一定是 1。我们用一根树枝两端连接的结点的编号来描述一根树枝的位置。下面是一颗有 4 个树枝的树

```

2  5
 \ /
 3  4
 \ /
 1

```

现在这颗树枝条太多了，需要剪枝。但是一些树枝上长有苹果。给定需要保留的树枝数量，求出最多能留住多少苹果。

输入格式

第 1 行 2 个数，N 和 Q($1 \leq Q \leq N, 1 < N \leq 100$)。N 表示树的结点数，Q 表示要保留的树枝数量。接下来 N-1 行描述树枝的信息。每行 3 个整数，前两个是它连接的结点的编号。第 3 个数是这根树枝上苹果的数量。每根树枝上的苹果不超过 30000 个。

输出格式

一个数，最多能留住的苹果的数量。

样例输入

```
5 2
1 3 1
1 4 10
2 3 20
```

```
3 5 20
```

样例输出

```
21
```

解析：因为题目一给出就是二叉的，所以很容易就可以写出方程：

$$a(l,j) := \max(a(i.\text{left},k) + a(i.\text{right},j-k)), 0 \leq k \leq j$$

源程序代码：

由于比较简单便不给完全的代码了。

```
Function treedp(x,y:longint):longint;
```

```
Var l,j,k:longint;
```

```
Begin
```

```
J:=0;
```

```
For l:=0 to y do
```

```
begin
```

```
k:=treedp(b[x].l,l)+treedp(b[x].r,y-l);
```

```
if k>j then j:=k;
```

```
end;
```

```
treedp:=j;
```

```
End; {要再仔细想想. . . . . }
```

选课

[问题描述]

在大学里每个学生，为了达到一定的学分，必须从很多课程里选择一些课程来学习，在课程里有些课程必须在某些课程之前学习，如高等数学总是在其它课程之前学习。现在有 N 门功课，每门课有个学分，每门课有一门或没有直接先修课（若课程 a 是课程 b 的先修课即只有学完了课程 a，才能学习课程 b）。一个学生要从这些课程里选择 M 门课程学习，问他能获得的最大学分是多少？

输入:

第一行有两个整数 N, M 用空格隔开。 ($1 \leq N \leq 200, 1 \leq M \leq 150$)

接下来的 N 行, 第 $i+1$ 行包含两个整数 k_i 和 s_i , k_i 表示第 i 门课的直接先修课, s_i 表示第 i 门课的学分。若 $k_i=0$ 表示没有直接先修课 ($1 \leq k_i \leq N, 1 \leq s_i \leq 20$)。

输出:

只有一行, 选 M 门课程的最大得分。

样例:

输入:

```
7 4
2 2
0 1
0 4
2 1
7 1
7 6
2 2
```

输出:

```
13
```

解析:

这题比苹果树多了一个步骤就是把一棵普通树转化为二叉树。

读入数据时把二叉树建好: 第一个孩子作为父节点的左子树, 其它孩子作为第一个孩子的右子树。

$F(x, y)$: 表示节点 x 取 y 门课得最高学分, 则

$F(x, y) = \max(f(x.l, k-1) + x.v + f(x.r, y-k)) \quad k=0, 1, \dots, y$

$f(x.l, k-1) + x.v$ (课程 x 的学分): 表示选了课程 x , 左孩子选 $k-1$ 门课, 共 k 门课。

$f(x.r, y-k)$ 表示右孩子只能选 $y-k$ 门课。

标程中节点 -1 表示空节点, 0 是根节点, $1 \sim n$ 是 n 门可选课程的节点。

思考:若本题加上选那些课程可得到这个最大学分, 怎样修改程序?

实现:

怎么实现, 是在竞赛中的很重要的一个问题, 如果你想 **ac** 了这道题目的话, 你应该熟悉怎么把一棵树转化成二叉树, 完后怎么用递归的思想来实现动态规划。所以坚实的基础是很重要的东西, 如果没有了基础, 什么都是空中楼阁。

程序中已经边读边把二叉树建立好了。

源程序代码:

```
program bluewater;
type
tree=record
    l,r,k:longint;
end;
var
s:string;
i,j,k,l:longint;
n,m:longint;
```

```

a:array[0..200] of tree;
b:array[-1..200,0..150] of integer;
f:array[0..200] of longint;

procedure treedp(x,y:longint);
var i,j,k,l:longint;
begin
if b[x,y]>=0 then exit;
treedp(a[x].r,y);{只有右子树的情况}
j:=b[a[x].r,y];
for k:=1 to y do{左右子树都有的情况}
begin
treedp(a[x].l,k-1);
treedp(a[x].r,y-k);
i:=b[a[x].l,k-1]+b[a[x].r,y-k]+a[x].k;
if i>j then j:=i;
end;
b[x,y]:=j;
end;

begin
readln(s);
assign(input,s);reset(input);
readln(n,m);
fillchar(f,sizeof(f),0);
for i:=0 to n do
begin a[i].l:=-1;a[i].r:=-1;a[i].k:=-1;end;
{build tree}
for i:=1 to n do
begin
readln(k,l);
a[i].k:=l;
if f[k]=0 then a[k].l:=i
else a[f[k]].r:=i;
f[k]:=i;
end;
{bianjie}
for i:=-1 to n do
for j:=-1 to m do
if (i=-1) or (j=0) then b[i,j]:=0 else b[i,j]:=-1;
{tree dp}
treedp(a[0].l,m);
{output}
writeln(b[a[0].l,m]);
end.

```

Tju1053 技能树

Problem

玩过 **Diablo** 的人对技能树一定是很熟悉的。一颗技能树的每个结点都是一项技能，要学会这项技能则需要耗费一定的技能点数。

只有学会了某一项技能以后，才能继续学习它的后继技能。每项技能又有着不同的级别，级别越高效果越好，而技能的升级也是需要耗费技能点数的。

有个玩家积攒了一定的技能点数，他想尽可能地利用这些技能点数来达到最好的效果。因此他给所有的级别都打上了分，他认为效果越好的分数也越高。现在他要你帮忙寻找一个分配技能点数的方案，使得分数总和最高。

Input

该题有多组测试数据。

每组测试数据第一行是一个整数 n ($1 \leq n \leq 20$)，表示所有不同技能的总数。

接下来依次给出 n 个不同技能的详细情况。

每个技能描述包括 5 行。

第一行是该技能的名称。

第 2 行是该技能在技能树中父技能的名称，名称为 **None** 则表示该技能不需要任何的先修技能便能学习。

第 3 行是一个整数 L ($1 \leq L \leq 20$)，表示这项技能所能拥有的最高级别。

第 4 行共有 L 个整数，其中第 l 个整数表示从地 $l-1$ 级升到第 l 级所需要的技能点数（0 级表示没有学习过）。

第 5 行包括 L 个整数，其中第 l 个整数表示从第 $l-1$ 级升级到第 l 级的效果评分，分数不超过 20。

在技能描述之后，共有两行，第 1 行是一个整数 P ，表示目前所拥有的技能点数。

接下来 1 行是 N 个整数，依次表示角色当前习得的技能级别，0 表示还未学习。这里不会出现非法情况。

Output

每组测试数据只需输出最佳分配方案所得的分数总和。

Sample Input

```
3
Freezing Arrow
Ice Arrow
3
3 3 3
15 4 6
Ice Arrow
Cold Arrow
2
4 3
10 17
Cold Arrow
None
3
3 3 2
```

15 5 2

10

0 0 1

[Sample Output](#)

42

Source

[浙江省 2004 组队赛第二试](#)

解析：这题是选课的加强版，但并难不倒我们

还是把一棵树转换为二叉树，完后从子节点到根节点作一次 **dp**，最后得到最优解

由于和上题很相像就不写方程了。

源代码程序：

```
program bluewater;
type
tree=record
    s,sf:string;
    l,r,m:longint;
    c:array[1..20] of longint;
    d:array[1..20] of longint;
end;
var
i,j,k,l,m,n:longint;
a:array[0..20] of tree;
b:array[0..20] of longint;
learn:array[0..20] of longint;
f:array[0..20,0..8000] of longint;

function treedp(x,y:longint):longint;
var i,j,k,l,max,o,p,q:longint;
begin
if f[x,y]<>-1 then begin treedp:=f[x,y];exit;end;
max:=treedp(a[x].r,y);
{learn>0}
if learn[x]>0 then
begin
for k:=0 to y do
begin
i:=treedp(a[x].l,k)+treedp(a[x].r,y-k);
if i>max then max:=i;
end;
end;
{learn=0}
l:=0;p:=0;i:=0;
for o:=1 to a[x].m do
begin
```

```

    if o>learn[x] then
        begin l:=l+a[x].c[o];p:=p+a[x].d[o];end;
    for k:=0 to y-l do
        begin
            i:=treedp(a[x].l,k)+treedp(a[x].r,y-l-k)+p;
            if i>max then max:=i;
        end;
    end;
    f[x,y]:=max;
    treedp:=max;
end;

```

```

function find(x:string):longint;
var i,j:longint;
begin
    for i:=0 to n do
        if a[i].s=x then break;
    find:=i;
end;

```

```

begin
    while not(eof(input)) do
        begin
            {input}
            readln(n);
            fillchar(a,sizeof(a),0);
            fillchar(b,sizeof(b),0);
            a[0].s:='None';
            for i:=1 to n do
                with a[i] do
                    begin
                        readln(s);
                        readln(sf);
                        readln(m);
                        for j:=1 to m do read(c[j]);readln;
                        for j:=1 to m do read(d[j]);readln;
                    end;
            readln(m);
            if m>8000 then m:=8000;
            for i:=1 to n do read(learn[i]);readln;
            {build binary tree}
            for i:=1 to n do
                begin
                    k:=find(a[i].sf);
                    if b[k]=0 then

```

```
begin b[k]:=i;a[k].l:=i;end
  else begin a[b[k]].r:=i;b[k]:=i;end;
end;
{bian jie}
for i:=0 to 20 do
for j:=0 to 8000 do
f[i,j]:=-1;
for i:=0 to 8000 do f[0,i]:=0;
{main}
writeln(treedp(a[0].l,m));
end;
end.
```